COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	07/31/26	**DATE SUBMITTED:**	08/05/26
NAME:	Prince Benjamin S. Villaruel	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 4 Stacks

1. PROCEDURE OUTPUT

What procedures did you perform?

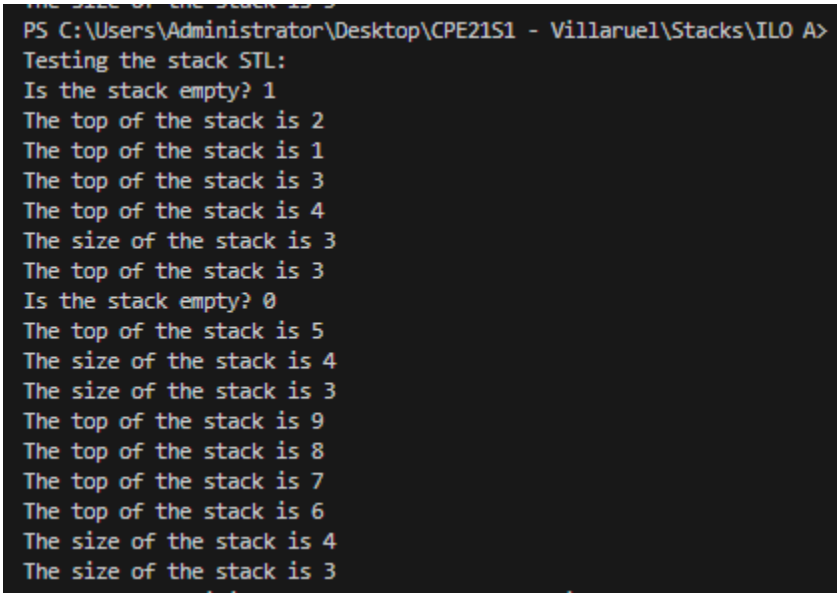
ILO A: Create a stack using the C++ STL

amain.cpp > main()

```cpp
#include <iostream>
#include <stack>

int main(){
    std::stack<int> stack1;
    std::cout<<"Testing the stack STL:"<<std::endl;

    //isempty
    std::cout<<"Is the stack empty? "<<stack1.empty()<<std::endl;
    //push
    stack1.push(2);
    std::cout<<"The top of the stack is "<<stack1.top()<<std::endl;
    stack1.push(1);
    std::cout<<"The top of the stack is "<<stack1.top()<<std::endl;
    stack1.push(3);
    std::cout<<"The top of the stack is "<<stack1.top()<<std::endl;
    stack1.push(4);
    std::cout<<"The top of the stack is "<<stack1.top()<<std::endl;
    //pop
    stack1.pop();
    std::cout<<"The size of the stack is "<<stack1.size()<<std::endl;
    std::cout<<"The top of the stack is "<<stack1.top()<<std::endl;
    //isempty
    std::cout<<"Is the stack empty? "<<stack1.empty()<<std::endl;
    //emplace
    stack1.emplace(5);
    std::cout<<"The top of the stack is "<<stack1.top()<<std::endl;
    //before pop
    std::cout<<"The size of the stack is "<<stack1.size()<<std::endl;
    stack1.pop();
    //after pop
    std::cout<<"The size of the stack is "<<stack1.size()<<std::endl;
    //swap
    std::stack<int> stack2;
    //push stack2
    stack2.push(9);
    std::cout<<"The top of the stack is "<<stack2.top()<<std::endl;
    stack2.push(8);
    std::cout<<"The top of the stack is "<<stack2.top()<<std::endl;
    stack2.push(7);
    std::cout<<"The top of the stack is "<<stack2.top()<<std::endl;
    stack2.push(6);
    std::cout<<"The top of the stack is "<<stack2.top()<<std::endl;
    //swapping'
    stack1.swap(stack2);
    //results
    std::cout<<"The size of the stack is "<<stack1.size()<<std::endl;
    std::cout<<"The size of the stack is "<<stack2.size()<<std::endl;
}
```



Observation

- I found that the STL stack handled all operations (push, pop, top, empty, emplace, and swap) correctly and required the least amount of work since I am just testing each operation and the memory automatically manages itself.

B.1. Stacks using Arrays

Welcome amain.cpp bmain1.cpp

C: > Users > Administrator > Desktop > CPE21S1 - Villaruel > Stacks > ILO B > bmain1.cpp > pop()

```cpp
#include <iostream>
const size_t maxCap = 100;
int stack[maxCap]; //stack max 100 elements
int top = -1, i, newData;

void push();
void pop();
void Top();
bool isEmpty();
void displayStack(); //display all elements

int main(){
    int choice;
    std::cout << "Enter number of max elements for new stack: ";
    std::cin >> i;

    while(true){
        std::cout << "Stack Operations: "<< std::endl;
        std::cout << "1. Push, 2. Pop, 3. Top, 4. isEmpty, 5. Display"<<std::endl;
        std::cin >> choice;

        switch (choice) {
            case 1:
                push();
                break;
            case 2:
                pop();
                break;
            case 3:
                Top();
                break;
            case 4:
                std::cout<<isEmpty()<<std::endl;
                break;
            case 5:
                displayStack();
                break;
            default:
                std::cout<<"Invalid Choice."<<std::endl;
                break;
        }
    }
    return 0;
}
```

```cpp
bool isEmpty(){
    if (top == -1)
        return true;
    return false;
}

void push(){
    //check if full
    if (top == i - 1){
        std::cout<<"Stack Overflow." <<std::endl;
        return;
    }
    std::cout<<"New Value: ";
    std::cin >> newData;
    stack[++top] = newData;
}

void pop(){
    //check if empty
    if (isEmpty()) {
        std::cout << "Stack Underflow." <<std::endl;
        return;
    }
    //Display the top value
    std::cout<<"Popping: "<<stack[top]<<std::endl;
    //decrement
    top--;
}

void Top(){
    if (isEmpty()){
        std::cout<<"Stack is Empty."<<std::endl;
        return;
    }
    std::cout << "The element on the top of the stack is "
        << stack[top] << std::endl;
}

//display all elements
void displayStack(){
    if (isEmpty()) {
        std::cout<<"There is nothing to display." << std::endl;
        return;
    }
    std::cout << "All stack elements: "<<std::endl;
    for (int idx = top; idx >= 0; idx--){
        std::cout << stack[idx] << std::endl;
    }
}
```

```
PS C:\Users\Administrator\Desktop\CPE21S1 - Villaruel\Stacks\ILO A>
Enter number of max elements for new stack: 5
Stack Operations:
1. Push, 2. Pop, 3. Top, 4. isEmpty, 5. Display
1
New Value: 1
Stack Operations:
1. Push, 2. Pop, 3. Top, 4. isEmpty, 5. Display
1
New Value: 2
Stack Operations:
1. Push, 2. Pop, 3. Top, 4. isEmpty, 5. Display
3
The element on the top of the stack is 2
Stack Operations:
1. Push, 2. Pop, 3. Top, 4. isEmpty, 5. Display
4
0
Stack Operations:
1. Push, 2. Pop, 3. Top, 4. isEmpty, 5. Display
2
Popping: 2
Stack Operations:
1. Push, 2. Pop, 3. Top, 4. isEmpty, 5. Display
5
All stack elements:
1
Stack Operations:
1. Push, 2. Pop, 3. Top, 4. isEmpty, 5. Display
6
Invalid Choice.
Stack Operations:
1. Push, 2. Pop, 3. Top, 4. isEmpty, 5. Display
```

Observation

- I got the array implementation working correctly after once I fixed the displayStack loop condition, but I noticed it relies on a fixed capacity and needs manual checking .

B.2. Stacks using Linked Lists

bmain2.cpp | stack_ll.h

C: > Users > Administrator > Desktop > CPE21S1 - Villaruel > Stacks > ILO B > stack_ll.h > displayALL<T>()

```cpp
#ifndef STACK_LL_H
#define STACK_LL_H

//class node
template <typename T>
class Node{
    public:
    T data; //storing
    Node* next; //point to next node
};

template <typename T>
Node<T> *head = nullptr;

//operations
//push
template <typename T>
void push(T newData){
    //create
    Node<T>* newNode = new Node<T>();
    newNode->data = newData;

    //points to current top then becomes that top
    newNode->next = head<T>;
    head<T> = newNode;
}

//pop
template <typename T>
T pop(){
    //check if empty
    if(head<T> == nullptr){
        std::cout<<"Stack Underflow."<<std::endl;
        return T();
    }

    //save
    Node<T>* temp = head<T>;
    //store
    T tempVal = temp->data;

    //move the head
    head<T> = head<T>->next;
    delete temp;
    return tempVal;
}

//isEmpty
template <typename T>
bool isEmpty(){
    return head<T> == nullptr;
}

//top
template <typename T>
void Top(){
    //error catching
    if(isEmpty<T>()){
        std::cout<<"Stack Underflow."<<std::endl;
        return;
    }
    //display
    std::cout<<"Top of stack: "<<head<T>->data << std::endl;
}


//display all
template <typename T>
void displayALL(){
    if (head<T> == nullptr){
        std::cout << "Stack Underflow.\n";
        return;
    }
    std::cout<<"Display the Stack: \n";
    Node<T>* temp = head<T>;
```

bmain2.cpp | stack_ll.h

C: > Users > Administrator > Desktop > CPE21S1 - Villaruel > Stacks > ILO B > bmain2.cpp > main()

```cpp
#include<iostream>
#include "stack_ll.h"

int main(){

    std::cout<< "PUSHING 'c' 'p' 'e' ONTO THE STACK\n";
    //push
    push('c');
    push('p');
    push('e');

    //display the current top
    Top<char>();
    displayALL<char>();
    std::cout<<"Popping all the elements\n";

    while(!isEmpty<char>()){
        std::cout<<" Popped: "<< pop<char>() <<std::endl;
    }

    std::cout<<"Testing the Underflow\n";
    pop<char>();

    return 0;
}
```

```
PS C:\Users\Administrator\Desktop\CPE21S1 - Villaruel\Stacks\ILO B
n2 } ; if ($?) { .\bmain2 }
PUSHING 'c' 'p' 'e' ONTO THE STACK
Top of stack: e
Display the Stack:
e
p
c

Popping all the elements
 Popped: e
 Popped: p
 Popped: c
Testing the Underflow
Stack Underflow.
```

Observation

- I found this one harder than the array version. The linked list stacks grows dynamically with no limit, but it requires me to carefully manage memory manually with the new and delete functions.

COMPUTER ENGINEERING

QUEZON CITY

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

A. Stack using Arrays

Expression	Valid? (Y/N)	Output (Console Screenshot)	Analysis
(A+B)+(C-D)	Y	Balance Checker (Array): Expression: (A+B)+(C-D) -> Valid (Y)	Every (is matched by a corresponding) in order.
((A+B)+(C-D)	N	Expression: ((A+B)+(C-D) -> Invalid (N)	One (is left unmatched on the stack.
((A+B)+[C-D])	Y	Expression: ((A+B)+[C-D]) -> Valid (Y)	All opening and closing symbols nest and match correctly.
((A+B]+[C-D]}	N	Expression: ((A+B]+[C-D]) -> Invalid (N)	The] does not match { on top of the stack.

Welcome array.cpp

array.cpp > isBalancedArray(const std::string &)

```cpp
#include <iostream>
#include <string>

//helpers
bool isMatchingPair(char opening, char closing){
    if(opening=='('&& closing== ')') return true;
    if(opening=='['&& closing== ']') return true;
    if(opening=='{'&& closing== '}') return true;
    return false;
}
bool isOpening(char c){return c=='(' || c=='{' || c=='['; }
bool isClosing(char c){return c==')' || c=='}' || c==']'; }

//stack using arrays
bool isBalancedArray(const std::string &expr){
    const int maxCap = 100;
    char stackArr[maxCap];
    int top = -1;

    for(char c : expr){
        if(isOpening(c)){
            stackArr[++top] = c; //push
        }
        else if(isClosing(c)){
            if(top == -1) return false;
            char popped = stackArr[top--];
            if(!isMatchingPair(popped, c)) return false; //mismatch = error
        }
    }
    return top == -1;
}
int main(){
    std::string testCases[] = {
        "(A+B)+(C-D)",
        "((A+B)+(C-D)",
        "((A+B)+[C-D])",
        "((A+B]+[C-D])"
```

B. Stack using Linked List

Expression	Valid? (Y/N)	Output (Console Screenshot)	Analysis
(A+B)+(C-D)	Y	Balance Checker (Linked List): Expression: (A+B)+(C-D) -> Valid (Y)	Different implementation same results
((A+B)+(C-D)	N	Expression: ((A+B)+(C-D) -> Invalid (N)	Different implementation same results
((A+B)+[C-D])	Y	Expression: ((A+B)+[C-D]) -> Valid (Y)	Different implementation same results
((A+B]+[C-D]}	N	Expression: ((A+B]+[C-D]) -> Invalid (N)	Different implementation same results

Welcome | array.cpp | main.cpp | balance_checker_ll.h

balance_checker_ll.h > ...

```cpp
#ifndef BALANCE_CHECKER_LL_H
#define BALANCE_CHECKER_LL_H
#include <string>

//helpers
bool isMatchingPair(char opening, char closing){
    if(opening=='('&& closing== ')') return true;
    if(opening=='['&& closing== ']') return true;
    if(opening=='{'&& closing== '}') return true;
    return false;
}
bool isOpening(char c){return c=='(' || c=='{' || c=='['; }
bool isClosing(char c){return c==')' || c=='}' || c==']'; }

//stack using LL

struct Node{
    char data;
    Node* next;
};

inline bool isBalancedLinkedList(const std::string &expr){
    Node* head = nullptr;

    for(char c : expr){
        if(isOpening(c)){
            //push
            Node* newNode = new Node();
            newNode->data = c;
            newNode->next = head;
            head = newNode;
        }
        else if(isClosing(c)){
            if(head == nullptr) return false;

            //pop
            Node* temp = head;
            char popped = temp->data;
            head = head->next;
            delete temp;

            if(!isMatchingPair(popped, c)) return false;
        }
```

main.cpp > main()

```cpp
#include <iostream>
#include "balance_checker_ll.h"

int main(){
    std::string testCases[] = {
        "(A+B)+(C-D)",
        "((A+B)+(C-D)",
        "((A+B)+[C-D])",
        "((A+B]+[C-D])"
    };

    std::cout <<"Balance Checker (Linked List): "<<std::endl;
    for(const std::string &expr : testCases){
        std::cout << "Expression: " << expr << " -> "
            << (isBalancedLinkedList(expr) ? "Valid (Y)" : "Invalid (N)")<<std::endl;
    }
    return 0;
}
```

Observation

Array

- The array based checker correctly highlighted all four test expressions, showing that a simple push, pop, compare routine is enough to validate nested symbol pairs without needing dynamic memory.

Linked List

- The linked list based checker highlighted the same results to the array version on all four expressions, confirming that as long it is the same logic it produces the same results despite different stack implementations.

Conclusion

Working through ILO A up into supplementary activity, I realize that a stack's last in first out stayed consistent no matter what implementation I used. ILO A paved the way for the basic functions in stack, since the STL let me get comfortable without worrying about memory. ILO B.1 and B.2 then had me build the same functions using an array and a linked list, which showed the pros and cons of each. Finally, the supplementary activity let me apply everything by using both the array and linked list stacks to solve a problem. Getting identical results from both confirmed that despite the implementation, the concept of stack remains the same.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Claude & Grammarly**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar correction & Explanation of concepts**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I used Grammarly to fix some of the grammar in my observations and Claude to help explain concepts that I was trying to add in my program.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.


T.I.P
TECHNOLOGICAL INSTITUTE OF THE PHILIPPINES
COMPUTER ENGINEERING
QUEZON CITY